

부트로더 u-boot #1

글쓴이 : [고도리](#) (2004 년 07 월 02 일 오전 01:42) 읽은수: 8,715

[[임베디드강좌/윤덕배](#) ]

0. 부트로더 분석

아시는 분들한테는 별로 대단한 것도 아니겠지만, 아직 이쪽에 대해서 분위기가 익숙하지 않으신 분들한테는 조금이나마 도움이 될거라고 생각을 하면서 그리고 제 자신이 썼던글을 정리하는 겸해서 부트로더에 대한 글을 정리할까 합니다.

부트로더란 간단하게 특정 cpu 에 OS 혹은 어떤 프로그램을 돌릴 수 있도록 cpu 가 동작하는데 필요한 아주 기초적인 부분이나 ROM(or flash), RAM, UART 등의 기본적인 디바이스들을 동작할 수 있게 만드는 프로그램입니다.

부트로더는 직접 작성을 해도 남이 작성을 해 놓은 것을 써도 됩니다.

여기서 분석할 것은 u-boot 인데, 이것을 선택한 이유가

첫째: PowerPC 할때 ppcboot 를 써서 많이 익숙하다.

둘째: 발전을 해가면서 다양한 platform 에 포팅되어 있습니다(ppc, arm, mips, x86...)

셋째: 코드가 깔끔하고 좋습니다. 즉, 한번 공부해볼만한 가치가 있다고 생각합니다.

넷째: 가장 중요한것...제가 이것밖에 안써봐서요...^^

자 그럼 시작.....!!!

1. u-boot 란?

README 에서 발췌

"

This directory contains the source code for U-Boot, a boot loader for Embedded boards based on PowerPC and ARM processors, which can be installed in a boot ROM and used to initialize and test the hardware or to download and run application code.

The development of U-Boot is closely related to Linux: some parts of the source code originate in the Linux source tree, we have some

header files in common, and special provision has been made to support booting of Linux images.

Some attention has been paid to make this software easily configurable and extendable. For instance, all monitor commands are implemented with the same call interface, so that it's very easy to add new commands. Also, instead of permanently adding rarely used code (for instance hardware test utilities) to the monitor, you can load and run it dynamically.

"

그냥 간단하게 말하면 **PowerPC** 와 **ARM** 에 기반을 둔 임베디드 보드를 위한 부트로더입니다.
(영어 능력의 한계...--;)

원래 **ppc** 를 위한 **ppcboot** 란 프로젝트가 있었습니다. 이와는 한편에서 **armboot** 란 프로젝트가
있었습니다. 각각 **ppc** 계열과 **arm** 계열의 **cpu** 를 위한 부트로더였는데, 두개를 통합하면서 **u-boot** 가 만들어 졌습니다.

u-boot 를 많이 쓰는 이유는 굉장히 강력하고 그나마 쉬운(?) 환경 설정에 있습니다.
많이 쓰이는 웬만한 **cpu** 에 대한 **evaluation board** 에 대한 기본 **sample** 코드가 있으므로
그것을 보면서 약간의 수정을 가하기만 하면 동작되도록 만들어 졌습니다.

만일 삼성 **s3c2410** 이란 **cpu** 를 쓴 **smdk2410** 보드의 경우 그냥 컴파일 해서 올리기만
network 까지 아무 문제없이 동작하도록 만들어져 있습니다.

그리고, **ppc** 를 쓰건 **arm** 을 쓰건 같은 명령어 체계를 사용하므로, 밑의 **hw** 단은
틀리더라도
상위단의 부트로더 명령같은 경우는 같습니다. 그러므로, 다른 **platform** 에 적용을
하더라도
큰 어려움 없이 쉽게 접근할 수 있습니다.

원래 만든 회사의 말과 다운로드 받는 법

"

U-Boot is Open Source Firmware for Embedded PowerPC, ARM, MIPS and x86 processors.

The U-Boot project is hosted at Sourceforge, where you can find the project home page:

<http://sourceforge.net/projects/u-boot>

The current version of the U-Boot source code can be retrieved from the CVS repository at Sourceforge using anonymous (pserver) CVS.

Press the "Enter" key when asked for the password for user "anonymous":

```
cvs -d:pserver:anonymous@cvs.u-boot.sourceforge.net:/cvsroot/u-boot login
```

```
cvs -z6 -d:pserver:anonymous@cvs.u-boot.sourceforge.net:/cvsroot/u-boot co -P  
modulename
```

Official releases of U-Boot are also available through FTP.
Compressed tar archives can be downloaded from the directory
<ftp://ftp.denx.de/pub/u-boot/>.

"

다운로드는 간단합니다. 위의 웹사이트가면 있습니다. 개발 버전은 밑의 ftp 에 있고요.

현재 버전은 1.1.1 입니다만 기본은 1.0.0 으로 하고 코드는 제가 arm 용으로 분석했던 당시의

0.4.0 을 섞어서 설명할겁니다. 뭐 크게 달라진것은 없으니 충분히 할만할겁니다.

2. u-boot 트리구조

자 압축을 풀면....u-boot-1.0.0 이란 디렉토리 밑에 다음과 같은 트리가 생길겁니다.
물론 많이 줄인편인데...그래도 굵직한 녀들은 남겼습니다...^^

/board ---/smdk2410

/smdk2400

...

/common

/cpu ---/cpu/74xx_7xx

/cpu/arm720t

/cpu/arm920t

/cpu/arm925t

/cpu/arm926ejs

/cpu/at91rm9200

/cpu/i386

/cpu/ixp

/cpu/mips

/cpu/mpc5xx

/cpu/mpc5xxx

/cpu/mpc824x

/cpu/mpc824x/drivers

/cpu/mpc824x/drivers/dma

/cpu/mpc824x/drivers/epic

/cpu/mpc824x/drivers/i2c

/cpu/mpc824x/drivers/i2o

/cpu/mpc8260

/cpu/mpc85xx

/cpu/mpc8xx

/cpu/nios

/cpu/ppc4xx

/cpu/pxa

/cpu/sa1100

/disk

/doc

/drivers

/dtb

/examples

/fs ---/fat

/fdos

/jffs2

/include ---/include/asm-arm
/include/asm-arm/arch-arm920t
/include/asm-arm/arch-arm925t
/include/asm-arm/arch-arm926ejs
/include/asm-arm/arch-at91rm9200
/include/asm-arm/arch-ixp
/include/asm-arm/arch-pxa
/include/asm-arm/arch-sa1100
/include/asm-arm/proc-armv
/include/asm-i386
/include/asm-i386/ic
/include/asm-mips
/include/asm-nios
/include/asm-ppc
/include/bedbug
/include/configs
/include/galileo
/include/jffs2
/include/linux
/include/linux/byteorder
/include/linux/mtd
/include/pcmcia

/lib_arm
/lib_generic
/lib_i386
/lib_mips
/lib_nios
/lib_ppc
/net
/post
/rtc
/tools

컴파일 하는 방법들은 나중에 실무를 설명할때 설명하고요, 간단하게 이번엔 트리구조만 설명드리겠습니다.(....키보드 치기 싫어서...ㅋㅋ)

- "cpu" directory

각종 u-boot 에서 지원하는 cpu 에 대한 startup 코드(cpu 초기화)와 serial, clock, timer 등등의 cpu specific 한 코드들이 있는 디렉토리입니다.

예를들어 삼성의 s3c2400/s3c2410 의 경우는 arm920t, TI 의 omap 같은 경우는 arm925t, arm926ejs(맞나?..^^) 디렉토리에 있습니다.

만일 ppc 같은 경우 예를들어 내장 이더넷과 pci host bridge 를 갖고 있는 ppc405 의 경우는 ethernet 코드와 pci 코드가 여기에 있습니다.

- "board" directory

cpu 디렉토리에 있는 cpu 들로 만들어진 현재 판매되는 유명한편에(?) 속하는 상용 evaluation

에 대한 코드들이 있는 곳입니다.

뭐 삼성 s3c2410 cpu 를 쓴 유명한 보드는 smdk2410 이니 당연히 고런게 있겠지용.
pxa255 의 경우는 lubbock 보드가 될테고요(pxa250 인가?..^^)

보통은 보드에 밀접한 코드들이 있습니다. 예를 들어, 보드 초기화 코드, memory bank 설정코드

flash 코드, 부트로더가 dram 에 위치해야하는 relocation address 를 기록한 config.mk, 전체코드의 배치를 지정하는 u-boot.lds 라는 링커 스크립트 파일까지 있습니다.

- common

모든 보드에 기본적인 명령어와 main loop 파일들이 들어 있음

- include

뭐 당연히 include directory 입니다. 해당 platform 에 대한 코드는 include/asm-arm 같은식으로

존재합니다. 여기서 중요한 녀석은 u-boot.h 인데...여기에 board description structure 가 존재합니다. arm 의 경우는 부트로더에서만 거의 쓰이지만 ppc 의 경우는 이 구조체를 kernel 에서

사용하는 board information 구조체와 같은 형태로 만들어 줘야합니다.

그래야 부트로더에서 커널로 모든 정보를 한방에 넘길 수 있습니다.

arm 은 왜 안그러냐 하면 arm 은 부트로더에서 커널로 parameter 를 전달할때 ram 의 특정번지
(보통은 dram physical start address + 0x100 위치)에 tag parameter 형태로 건네주기
때문에
보통은 부트로더에서만 해당 구조체를 씁니다.

ppc 는 해당 구조체의 포인터를 커널을 호출할때 넘겨줍니다.(맞나?....^^, 코드를 다시
봐야겠네요)

- include/configs

아주 중요한 디렉토리입니다. 각 보드에 대한 설정파일들이 있습니다. 물론 .h 의 형태죠
ex> smdk2410 보드의 경우 smdk2410.h 입니다.

즉, [보드이름].h 의 형태입니다.

내용물은 나중에 자세하게 날잡아서 설명하기로 하고요.

중요한 사실인 "모든 설정을 여기서 한다"....만 알고 계십시오

- lib_arm

u-boot 의 arm 쪽 메인코드들이 있습니다. startup 코드에서 초기화가 다 끝나면 여기
디렉토리의
board.c 인가로 점프할겁니다...

- drivers

뭐 network 등등의 driver 들이 있습니다. 참고로 arm 같은 경우에 많이 쓰이는
cs8900 등이 있죠.

- example

뭐 신경끄세요..

- tools

중요한 디렉토리입니다..ㅎㅎ, u-boot 는 커널이미지를 다른 방법으로 만들어줘야합니다.
왜냐하면 리눅스의 zImage 형태의 self-extract 코드를 없애고 u-boot 자체내에서 커널이

압축이 되었다면 **c** 코드로 압축을 풀어서 그냥 제어권을 넘겨주게 됩니다.

이때 **u-boot**에서는 커널의 앞쪽에 특별한 **header**를 넣어서 해당정보를 얻어오는데(압축/비압축, 리눅스커널/BSD 커널, 커널의 **load address...**등등) 그 헤더를 맵글어주는 **mkimage** 란 녀석이 여기에 있습니다.

나머지는 **Makefile** 정도인데...뭐 이거는 당에 컴파일하는 법 설명할때 대충 후딱하고 넘어가죠

당에는 간단한 컴파일 방법이나 떠들고, 바로 **configuration** 파일의 구조를 설명하고 코드 분석으로 들어갑니다...